JKSCI

Journal of The Korea Society of Computer and Information

Design and PoC Validation of a Secure On-Premises Meeting Intelligence Pipeline for Construction Enterprises

Jaehwan Kim[1] · Jaeseong Yun[2] · Jongseok Lee[3] · Kyungmin Kim[4]

[1]aSSIST, Seoul, Republic of Korea; [2]XAIKOREA Inc., Seongnam, Republic of Korea

[3]RAD CNS Co., Ltd., Republic of Korea; [4]Security Platform, Republic of Korea

First and Corresponding Author: Jaehwan Kim (kjh861213@naver.com)

[Abstract]

This paper develops and evaluates artifact validation and checkpoint recovery within an on-premises meeting-intelligence pipeline for construction enterprises. Using the same ASR model and decoding configuration, we compared full restart, syntax-based reuse, and reuse with combined integrity and execution-condition checks. The first 120 seconds of 16 AMI test recordings were processed in independent 30-second chunks. All 912 runs terminated: 48 no-fault controls and 864 fault-injection runs. Full restart and combined validation preserved the uninterrupted reference transcript in every fault-injection run, whereas syntax-based reuse returned incorrect transcripts in 96 text-corruption runs. After interruption following the third saved chunk, the mean of recording-level median recovery latencies was 26.19 seconds for both reuse policies and 83.52 seconds for full restart; the recording-level paired reduction for combined validation was approximately 57.3 seconds. These results show the role of content validation and valid-chunk reuse under the tested conditions. They do not establish whole-system security or accuracy on Korean construction meetings. The additional benefit of execution-condition checks requires a checksum-only comparison.

▸ Key words : On-Premises AI, Meeting Intelligence, Artifact Integrity, Checkpoint Recovery, Fault Injection

I Introduction

Systems that transcribe speech within an organization must handle interruption and re-execution of long-running jobs as well as recognition quality. When transcripts for some chunks remain available, reusing them can reduce computation. However, treating partial files, readable files with altered content, or files produced from different inputs or settings as valid checkpoints can allow a job to finish without preserving its transcript. Execution completion and transcript preservation must therefore be checked separately.

Whisper and WhisperX provide foundations for speech recognition and temporal alignment [1,2]. Recovery policies address a separate question: which artifacts produced before an interruption can be trusted, and which chunks must be recomputed? We examine whether the transcript of an uninterrupted reference run using the same model can be preserved after specified faults while reducing recomputation. Because recognition errors in the original output may also be preserved, we distinguish recovery preservation from recognition accuracy against a reference transcript.

Our earlier proof of concept (PoC) for construction-meeting processing integrated speech recognition, speaker diarization, summarization, storage, and key management. Its design principles addressed data and key boundaries, artifact completeness, checkpoint recovery, and separation of evidence levels. This study develops the artifact-completeness and recovery principles into concrete decisions about transcript-chunk reuse and evaluates their recovery costs. We validate measurable recovery behavior within the existing design; we do not propose a new speech model or establish the security of the entire pipeline. The experiments use public English meeting recordings, while applicability to Korean construction meetings remains a subject for further validation.

We address two research questions (RQs). RQ1 asks how transcript preservation differs across reuse policies under specified file corruption. RQ2 asks how recomputation and time vary with fault type and interruption position among runs that preserve the transcript. Both questions are analyzed using the completed E1 experiment. The incremental benefit of adding execution-condition checks to hash checks, and differences from existing workflows, are treated as separate validation questions.

This is an empirical systems study comparing recovery policies for speech transcription processed in independent chunks. Using the same audio, model, and fault conditions, we jointly analyze transcript preservation and computational cost across policies. The results explain the consequences of deciding reuse solely from file readability and the cost changes associated with retaining valid chunks. Identifiers for inputs, models, and code, together with execution records that include failures, support traceability of the comparison.

Our interpretation separates validation information from cost. Changes to output content can be detected by comparison with a stored hash, whereas a valid artifact from another job requires a separate check of its suitability for the current input and settings. E1 combines hash and execution-condition checks, so the observed improvements are reported as outcomes of the combined policy. Comparisons that isolate each check's incremental benefit and differences from existing implementations are reserved for subsequent validation.

II Related Work and Study Scope

1 Speech Processing and Recovery Preservation

Whisper is an automatic speech recognition (ASR) model that learns audio-transcript relationships from large-scale weakly supervised data and converts speech into text [1]. WhisperX combines transcription with alignment to locate when words were spoken [2], while pyannote.audio addresses diarization by grouping speech segments by speaker [3]. Speech-processing research separately evaluates what was said, when it was said, and who said it. Our concern is whether the generated transcript remains unchanged after job interruption and restart. We fix the recognition model and processing chunks and compare only recovery policies.

2 Checkpoint Recovery and Fault Injection

Checkpointing enables computation to resume from saved intermediate state instead of repeating it from the beginning. In their survey of checkpoint- and log-based recovery, Elnozahy et al. identify the consistency of recovery state and the extent of repeated computation as central issues [4]. We save each completed audio-chunk transcript as an application-level checkpoint. If three of four chunks remain valid, only the last chunk needs processing. Recalculation is necessary, however, when the retained output is corrupt or was generated under conditions incompatible with the current input. We therefore assess suitability for reuse as well as the existence of stored output.

Fault injection deliberately introduces interruption or corruption to observe detection and recovery behavior. Hsueh et al. describe an experimental approach that jointly designs injection targets, mechanisms, and observation methods [5]. Instead of waiting for naturally occurring faults, we apply three fault types to the same audio. Stop-only interruption examines reuse of valid results; JSON truncation examines handling of unreadable files; and content modification examines validation of readable content. Comparing policies under the same conditions links outcome differences to their checks. Equal run counts across conditions provide a controlled comparison and do not represent the frequency of faults in an operational service.

3 Execution-Condition-Based Caching and Crash Consistency

Caching stores computed results for reuse by the same task. The key question is whether a stored result corresponds to the task currently requested. Bazel's remote cache distinguishes result metadata associated with inputs and an execution specification from storage of output content [6]. For example, even if file content is unchanged, a transcript cannot be reused when the input audio has changed. We apply this established principle to transcript chunks, separately checking content preservation and compatibility of generation conditions.

Nextflow, introduced by Di Tommaso et al., supports computational reproducibility by managing execution environments and workflows [7]. Cache reuse depends on task hashes incorporating inputs, scripts, containers, and related information, together with stored outputs [8]. DVC similarly manages stage re-execution and output restoration using declared dependencies and parameters [9]. The experimental data-lineage feature introduced in Nextflow 25.04 records relationships between output checksums and the tasks that produced them [10]. Snakemake's between-workflow cache also identifies results using inputs, parameters, and the software environment [11]. Provenance recording, reuse decisions, and output-content validation serve different roles depending on implementation and configuration; feature descriptions and actual fault responses must therefore be compared separately.

Research on interrupted storage examines the relationship between requested writes and the state recoverable after a crash. Pillai et al. show that

application update procedures depend on filesystem persistence properties and that these differences affect crash consistency [12]. From this perspective, interruption between updates to a transcript file and its validation record can leave the two at different points in time. E1 observes interruption after a chunk-save acknowledgment, whereas a separate file experiment observes process termination between individual storage steps. Together, these experiments clarify the roles of reuse decisions and storage ordering.

4 Meaning of On-Premises Deployment

On-premises processing is a deployment condition in which an organization can manage its models and data internally. NIST's zero-trust guidance emphasizes verifying trust in subjects and resources independently of physical or network location [13]. We evaluate recovery under the assumption that inputs, models, execution code, and the controller are trusted. Authentication, encryption, key separation, administrative privileges, and log management are security controls that must be addressed separately in an operational system.

These studies share an interest in the conditions for reusing stored results and the extent of computation repeated after interruption. Checkpointing research addresses recovery-state consistency; workflow research connects inputs and settings to outputs; and crash-consistency research examines the states left by storage operations. We apply these perspectives to speech transcription, measuring transcript preservation and cost under the same input and model. Our present contribution consists of comparative results for specified faults and reproducible evaluation records. Table 1 summarizes the comparisons needed to assess the independent effect of execution-condition checks and differences from existing tools.

Table 1. Scope of comparison with related work

Study or framework	Main focus	Relation to this study
Whisper and WhisperX [1,2]	Recognition and alignment	Whisper is E1's ASR foundation
pyannote.audio [3]	Speaker diarization	Diarization component of the earlier application
Bazel remote cache [6]	Task- and content-based reuse	Prior distinction between task compatibility and output integrity
Nextflow · DVC · Snakemake [8,9,10,11]	Reuse based on inputs, settings, and environment	Direct comparison pending; equivalent configurations required
Application crash consistency [12]	Storage updates and persistence	Distinguishes process-termination evidence from power-loss evidence
This study	Transcript preservation and recomputation cost	Policy comparison under specified faults using the same worker; failures retained

III Selective Recovery with Execution-Condition Validation

1 Tasks and Trust Boundaries

An artifact in this study is a file containing a transcript and related information produced by processing an audio chunk. Each recording is divided into four chunks. Artifact A_i for chunk i stores a transcript string (text), a list containing segment timing and related information (segments), and the execution conditions used to generate it (binding). Files use JavaScript Object Notation (JSON). A separate validation record L_i stores the file's SHA-256 (Secure Hash Algorithm 256-bit) hash and execution conditions. The hash is a digest used to detect changes in file bytes; it serves a different role from a digital signature authenticating the author of the validation record.

Execution conditions B_i comprise the chunk identifier, a hash of the input audio bytes, a hash of

the model weights, and a hash of the protocol file that includes inference settings. The controller calculates the expected conditions B_i* for the current task from the frozen input specification. It compares conditions stored in the artifact and validation record with these expected values to distinguish outputs from different inputs, models, or settings. Execution code and the container image are identified in a separate frozen manifest. Thus, the information used for E1 reuse decisions is related to, but distinct from, the information required to reproduce the experimental environment.

We assume that the controller, input specification, model, execution code, and validation records remain in their specified states. E1 corrupts output files while preserving validation records to determine whether the stored validation information detects artifact changes.

Fig. 1 shows how artifacts generated from audio chunks and execution conditions are used in recovery following a reuse decision. The three-dimensional depiction distinguishes logical components.

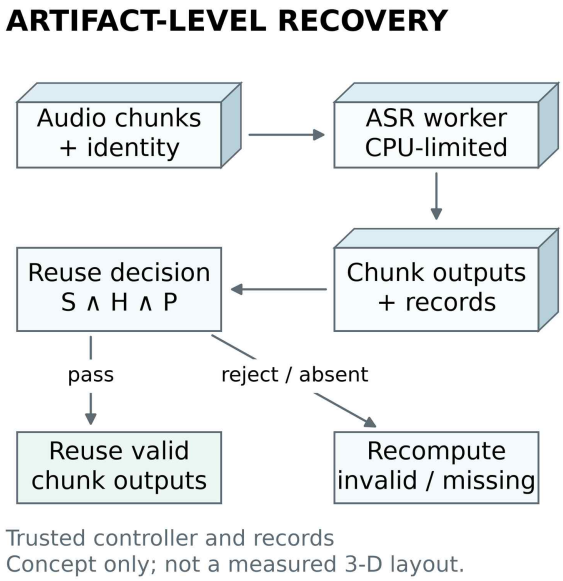


Fig. 1. Conceptual three-dimensional view of transcript-artifact generation, validation, and reuse.

2 Reuse Decisions

Reuse validation asks three questions. First, can the file be read and its required fields interpreted? The syntax condition S(A_i) is a minimal format check: it verifies that JSON can be parsed, text is a string, and segments is a list. An empty transcript for a silent chunk is permitted as a valid output. Second, has the file changed since it was stored? The integrity condition H(A_i,L_i) compares the hash of the current bytes with the value in the validation record. Third, is the file an output of the task currently being performed? The execution-condition match P(A_i,L_i,B_i*) compares the generation conditions in the artifact and record with those required by the current task.

For example, suppose that the current task requires model B, but a transcript produced by model A and its validation record remain available. If the JSON is readable and the file hash matches the record, syntax check S and content check H pass. Check P identifies the difference between the generating model and the required model. This example illustrates the roles of the checks and is distinct from E1's content-modification fault. The combined policy reuses chunks that satisfy all three conditions and reruns inference for the others.

The reuse rule is R_i = S(A_i) ∧ H(A_i,L_i) ∧ P(A_i,L_i,B_i*). Here, A_i is the transcript file, L_i is its validation record at storage time, B_i* specifies the execution conditions required by the current task, and ∧ means that every condition must hold. A valid artifact passes all three checks and is reused. Truncated JSON is selected for recomputation by S, while a changed transcript is selected by H. A file generated by another model together with its matching validation record can pass S and H, but P reveals the difference from the currently required model. These examples explain the decision rule; the effect of the last condition is evaluated separately from E1.

The decision process begins by reading the expected conditions for the current chunk. It then

applies the policy's checks, selects reusable chunks, infers the remaining chunks, and saves their outputs and validation records. The job ends when all chunks are ready. The evaluator compares the resulting transcript with the uninterrupted reference and records transcript preservation. Execution completion and evaluation outcomes are stored separately.

Full restart reruns inference for every chunk, whereas the syntax-based policy reuses outputs satisfying S. E1's combined-validation policy applies S, H, and P. The output-hash-only policy in the supplementary experiment applies S and H and is designed to assess the incremental effect of P through comparison with combined validation.

Fig. 2 maps the questions asked by the three checks to the policies compared experimentally. File readability, preservation of stored content, and suitability for the current execution are distinct properties.

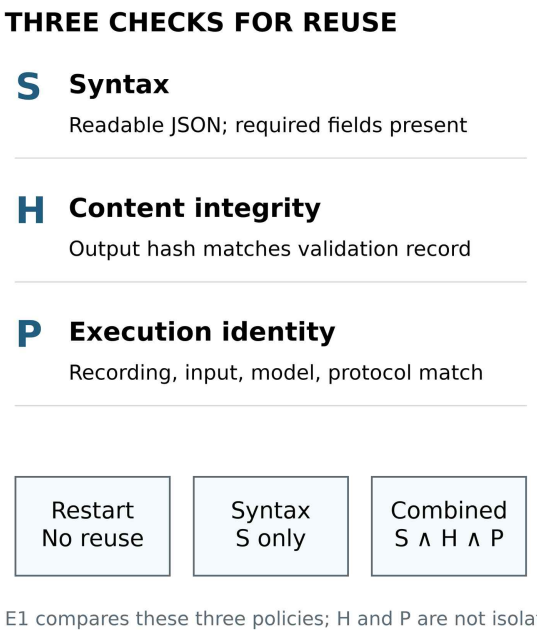


Fig. 2. Roles of syntax (S), content-hash (H), and execution-condition (P) checks and the policies compared in E1.

The syntax-based policy is a comparator for observing behavior under minimal file validation; full restart provides a cost baseline that recomputes every chunk. Under E1's content-modification condition, an output-hash check can detect the corruption. Isolating the effect of execution-condition check P requires comparison with an output-hash-only policy.

In this paper, committing a result means that a recovery job writes its result files and terminates. If that result differs from the reference transcript, it is counted as a correctness failure, corresponding to incorrect_commit in the raw records.

3 Storage and Result Commit

Understanding the state left by an interrupted write requires distinguishing updates to the output file from updates to its validation record. The E1 worker first writes the transcript to a temporary file and replaces the output file, then updates the validation record in the same way. Replacing each file and committing both files together are separate issues. An interruption between the two updates can leave a file and validation record from different points in time. E1 terminates the worker after acknowledgment of completed chunk processing; observations at the storage boundaries themselves are addressed in the separate file experiment.

After restart, the worker checks each chunk, reuses valid outputs, and performs inference for the remainder. Once all chunks have been collected, it records the job's termination state and compares the list of chunk-level transcript strings with the reference obtained by processing all chunks without a fault. Exact equality of this string list is the main experiment's correctness measure. The supplementary experiment extends the records to distinguish transcript-string equality from equality of the full structured output.

4 Interpretation of Decisions and Dependency Scope

The evaluation distinguishes suitability for reuse, preservation of the reference transcript, and recognition quality against a ground-truth transcript. Reuse suitability asks whether an output can be used for the current task; preservation asks whether it equals the uninterrupted result; and

ground-truth quality measures how accurately speech is recognized against a human reference. For example, returning the same sentence that the model originally misrecognized preserves the transcript but retains the recognition error. Timing information may also differ even when the sentence is identical. E1 therefore evaluates exact equality of chunk-level strings, while the supplementary experiment separately records structured-output equality and compliance with execution conditions.

The current implementation decodes each 30-second chunk in an independent context. Reprocessing one chunk therefore does not require transcripts from other chunks as inputs. The situation differs for long-form transcription conditioned on preceding text: changing an earlier chunk can affect later decoding. Such processing requires recording both the context used and the generation conditions, and verifying whether recomputation must extend to subsequent chunks. Our chunk-level recovery results should be interpreted under the independent-processing condition.

5 Operational Controls

Only one experimental job runs in an isolated container with read-only access to the model, code, and inputs. Central processing unit (CPU) usage is limited to 2 CPUs; the memory limit and combined memory-plus-swap limit are each 8 GiB. Graphics processing unit (GPU) access and external networking are disabled. Existing service states and available memory are monitored. If a previously healthy service deteriorates or resource criteria are violated, only the experimental job is stopped. Service monitoring is based on execution state.

IV Experimental Design and Analysis

1 Data and Fixed Conditions

The Augmented Multi-party Interaction (AMI) corpus provides public multi-party meeting recordings [14]. We used 16 test recordings specified by a split manifest at a fixed revision [15]. The first 120.000 seconds of each recording, including silence, were retained and divided into four consecutive 30-second chunks. Inputs were 16 kHz, single-channel, 16-bit pulse-code modulation (PCM) audio, using the original waveforms and fixed intervals without alteration. The unique input duration was 1,920 seconds.

The 16 recordings belong to four series: EN2002, ES2004, IS1009, and TS3003. Repeated runs of the same recording are technical replicates intended to observe variation in processing time. Hurlbert's discussion of pseudoreplication explains why the number of measurements must be distinguished from the number of independent samples [16]. We summarize results descriptively while accounting for dependence within recordings and series. We also acknowledge that existing results on the test data were inspected before follow-up experiments were designed.

The inference model was Whisper large-v3-turbo, running on a CPU with two threads. The language was English, temperature was 0, beam size was 1, and both fp16 and conditioning on preceding chunk text were disabled. SHA-256 identifiers for the model weights, execution image, protocol, input manifest, and schedule were frozen. These settings and identifiers define the fixed conditions for reproducibility.

The E1 worker directly calls load_model and transcribe in the Python OpenAI Whisper implementation. The experimental worker contains the ASR stage, with AMI Mix-Headset fixed as the input channel. The shared aarch64 host has 10 Cortex-X925 cores and 10 Cortex-A725 cores. The experimental container was limited to the equivalent of 2 CPUs, distinct from the full host capacity. The frozen container image identifies the execution environment, and hashes of the model, execution code, and input manifest are retained together. These identifiers are described in Appendix C.

2 Fault Matrix and Execution Order

The main experiment, E1, combines three policies, three fault types, two interruption positions, and three technical replicates. It comprises 16×3×3×2×3=864 fault-injection runs plus 16×3=48 no-fault controls, for a total of 912 runs. For each recording, an uninterrupted run processing all chunks provides the reference, and agreement of the no-fault controls was verified. The remaining runs followed a frozen schedule generated with seed 20261026.

After the worker saves the first or third chunk and sends a readiness acknowledgment, the controller terminates it with SIGKILL. In the stop-only condition, it restarts without further file changes. Truncation cuts the first chunk's JSON bytes to half their length. Content modification replaces the first chunk's text with CORRUPTED OUTPUT while preserving the JSON structure. The validation record remains unchanged in both corruption conditions. The intended SIGKILL is a fault-injection condition and is distinguished from unexpected execution failures such as abnormal worker termination or monitoring failure.

The procedure is as follows. First, all four chunks are processed without interruption to produce the reference transcript. Each fault-injection run performs actual inference from the beginning through the first or third chunk and waits for readiness acknowledgment after saving the output files and validation records. The controller terminates that worker and applies the specified corruption. It then launches a new worker with the same policy to determine reuse for each chunk and infer only those that cannot be reused. Finally, the four transcript strings are compared with the reference. Pre-interruption inference, intended interruption, corruption, recovery, and outcome evaluation are therefore distinct stages.

Table 2. Design of the main E1 experiment

Item	Specification
Input	First 120 seconds of each of 16 recordings; 4 series
Processing unit	Four 30-second chunks; no cross-chunk transcript conditioning
Policies	Full restart; syntax checks; combined validation (S+H+P)
Fault	Stop only; JSON truncation; text modification preserving syntax
Interruption position	After saving chunk 1 or 3
Replicates and runs	3 technical replicates per fault condition; 864 fault runs; 48 no-fault controls
Correctness	Exact chunk-string equality with the same recording's clean full-restart transcript
Limits	600-second limit per job; unexpected failures and partial results retained

Fig. 3 summarizes E1's fault-injection sequence. Post-interruption modification targets the first chunk's artifact, and the recovered transcript is compared with the uninterrupted reference.

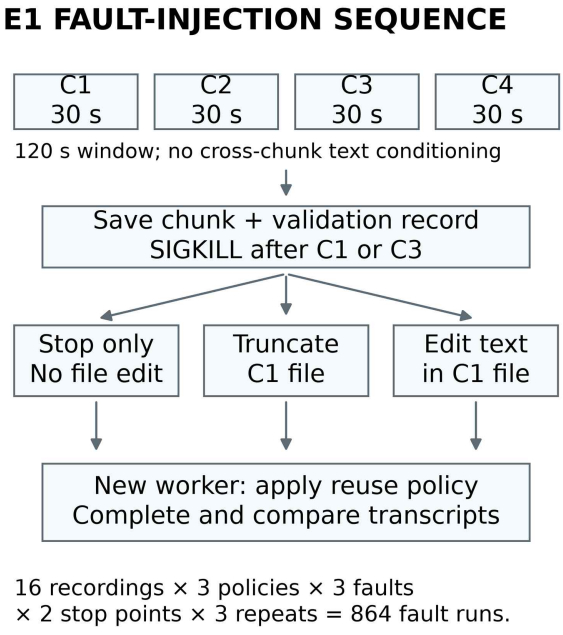


Fig. 3. E1 sequence of storage, interruption, modification, and recovery. C1–C4 are 30-second audio chunks; interruption occurs at C1 or C3 after the save acknowledgment.

The no-fault controls comprise 48 runs: one for each combination of 16 recordings and 3 policies. Fault injection comprises 16 recordings × 3 faults × 2 interruption positions × 3 policies × 3 replicates = 864 runs. The denominator for fault-recovery outcomes is 864.

3 Outcome Measures and Conditional Timing Analysis

Recovery correctness is evaluated against the transcript obtained by running the same input and model without a fault. Comparing the list of chunk-level strings with this reference reveals changes introduced during recovery. A normally terminated run with a different string list is counted as an incorrect commit. Execution failures, timeouts, and no-fault controls are recorded separately.

Recomputed audio duration C is the sum of the durations of chunks actually inferred after the fault. Post-fault latency L is measured from immediately before SIGKILL until immediately after the recovery worker terminates and its result and the reference transcript have been read. It includes corruption, worker restart, model loading, validation, inference, and storage. Total elapsed time T begins immediately before the first worker launch and ends at the same point, thus including pre-interruption inference. Writing the final result.json, container preparation, and waiting between batches are separate operational costs. Accordingly, L includes experimental control and result reading; applying it to user-perceived recovery time in a real service requires matching these measurement boundaries. We use L as the primary measure and report a post hoc analysis of recorded T in Table 5 and Appendix E.

Because recordings differ in speech content and processing difficulty, policies are compared in pairs on the same recording. Differences for policy a relative to full restart are ΔC=C_a-C_restart and Δ L=L_a-L_restart; negative values indicate cost reductions. We first subtract the paired runs for the same recording, fault, interruption position, and replicate, then take the median of the three differences for each recording. Finally, we average the recording-level values with equal weights. The median reduces the influence of a single large timing fluctuation. A recording is included only when both policies preserve the reference transcript in all three replicates; each condition in Table 4 includes 16 recordings. This aggregation order can yield a different result from subtracting separately computed policy medians.

Recovery policies must be assessed for both correctness and speed. Including runs that quickly return incorrect transcripts in a latency average can misrepresent recovery performance. We therefore report every correctness failure in Table 3 and compare timing only under correct-recovery conditions, stating excluded conditions and denominators. Reusing k chunks of 30 seconds reduces recomputed audio by 30k seconds. Actual time also depends on model loading, validation, storage, and host state. Reporting audio duration and time separately shows how reduced computation translates into reduced elapsed time.

4 Role of Supplementary Validation

After inspecting intermediate E1 results, we designed a supplementary experiment to isolate the effect of execution-condition check P by adding an output-hash-only comparator and conditions involving stale but valid artifact-record pairs. Supplementary designs and results are managed under separate protocols and paths, while all original frozen results and failures are preserved.

Supplementary validation is divided by the level of evidence sought. File storage-boundary experiment E3a uses 14 actual process terminations and 2 uninterrupted controls to examine how recovery policies select the files and validation records left at each interruption point. Development integration validation uses donors actually generated with another model or inference setting to verify that two ASR jobs proceed from interruption through recovery. The former checks file-selection rules,

while the latter checks implementation and execution cost; both are reported separately from E1. A separate supplementary main experiment has been frozen at 960 runs, comprising 64 no-fault and 896 fault-injection runs, plus 32 donor-generation runs, and is in progress. Effects across its policies will be evaluated separately after all results are available.

V Results

1 Execution Completion and Transcript Preservation

All 912 runs terminated normally, with no unexpected execution failures or timeouts. The 816 runs matching the reference transcript comprise 48 no-fault controls and 768 fault-injection runs. The remaining 96 of the 864 fault-injection runs committed incorrect transcripts.

Each fault-policy cell in Table 3 contains 96 runs across 16 recordings, two interruption positions, and three replicates. Under content modification, the syntax-based policy committed an incorrect transcript in all 96 runs. Because only text was modified after interruption, while JSON syntax and field types were preserved, its reuse condition was satisfied. These are correctness failures in which normally terminated jobs returned altered transcripts. Under stop-only interruption and truncation, all three policies preserved the transcript in all 96 runs per cell. Combined validation and full restart each preserved the transcript in all 96 content-modification runs as well.

Table 3. E1 outcomes by fault type and policy

Fault	Policy	Runs	Transcript matches	Incorrect commits
Stop only	Full restart	96	96	0
Stop only	Syntax checks	96	96	0
Stop only	Combined	96	96	0
Truncated JSON	Full restart	96	96	0
Truncated JSON	Syntax checks	96	96	0
Truncated JSON	Combined	96	96	0
Text edit	Full restart	96	96	0
Text edit	Syntax checks	96	0	96
Text edit	Combined	96	96	0
Total	Fault runs only	864	768	96

The 96 errors are repeated observations of the same corruption-policy combination. The central finding is that transcript preservation differed even among runs with the same normal termination status.

Fig. 4 illustrates the distinction between execution completion and transcript preservation. All 96 syntax-based runs under content modification terminated, but none preserved the reference transcript.

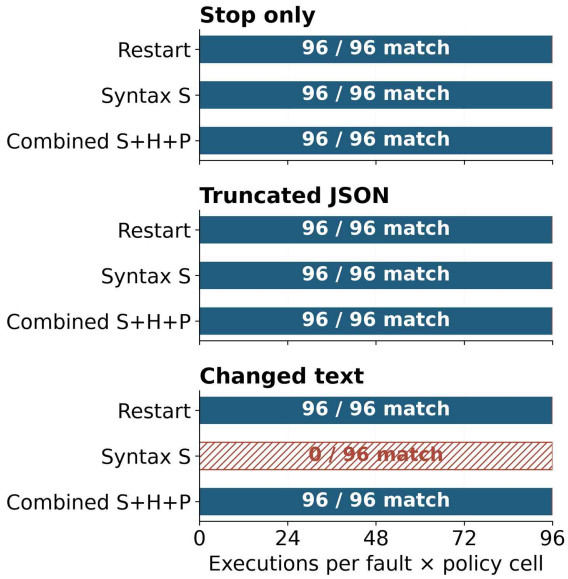


Fig. 4. Transcript preservation by policy in E1's 864 fault-injection runs. Each bar represents 16 recordings × 2 interruption positions × 3 replicates = 96 runs. Hatching denotes incorrect transcript commits; the 48 no-fault controls are excluded. The counts include technical replicates of the same recordings.

2 Recomputation and Latency in Correct Recovery

Table 4 compares combined validation with full restart. In every row, all three paired replicates for each of the 16 recordings satisfied the transcript-preservation condition. Following a stop-only interruption after the first saved chunk, combined validation computed only the remaining 90 seconds, saving 30 seconds of audio processing. After the third saved chunk, it computed only the last 30 seconds, saving 90 seconds. The corresponding latency differences were -16.061 and -57.312 seconds.

When truncation or content modification occurred at the first chunk, that chunk also had to be recomputed. All four chunks, or 120 seconds of audio, were therefore reprocessed. Recomputation savings were zero, and latency differences were -0.001 and +0.023 seconds, respectively, both close to zero. When the first chunk was corrupted after three chunks had been saved, the second and third chunks could be retained, saving 60 seconds of audio processing.

Table 4. Conditional differences between combined validation and full restart. n: eligible recordings.

Fault	Saved chunks	Recomp. (s)	ΔC (s)	ΔL (s)	n
Stop only	1	90	-30	-16.061	16
Stop only	3	30	-90	-57.312	16
Truncated JSON	1	120	0	-0.001	16
Truncated JSON	3	60	-60	-41.366	16
Text edit	1	120	0	+0.023	16
Text edit	3	60	-60	-41.439	16

ΔL is the equally weighted mean of recording-level medians of three paired differences. E1 full restart recomputes 120 seconds after the fault in every condition. The syntax-based policy achieved similar recomputation savings under stop-only interruption and truncation. Under content modification, however, all 16 recordings had correctness failures at both positions, so a latency contrast conditional on successful recovery was not defined.

Fig. 5 includes the syntax-based policy using the same aggregation rules as those used for combined validation in Table 4. Both reuse policies achieve similar savings under correct-recovery conditions. The significance of combined validation lies in the conditions under which reuse is permitted as well as the size of the savings.

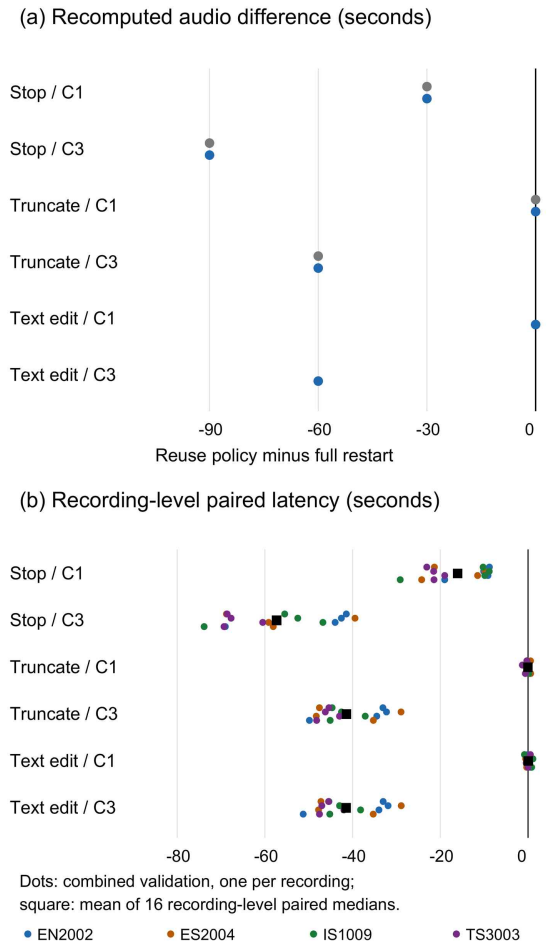


Fig. 5. Differences relative to full restart in (a) recomputed audio duration and (b) recording-level paired latency. In (a), gray denotes syntax-based reuse and blue denotes combined validation; correctness-failure conditions are excluded. In (b), points show recording-level medians of three paired differences for combined validation, and squares show the equally weighted mean across 16 recordings. Colors distinguish the four meeting series.

Table 5 presents a post hoc analysis of the existing E1 records using total elapsed time T, including pre-interruption cost. For stop-only interruption at position 3, post-fault latency L was 83.52 seconds for full restart and 26.19 seconds for each reuse policy. Corresponding T values were 145.20, 87.80, and 87.93 seconds. Savings shared by both reuse policies are associated with reuse of valid artifacts. The value of additional validation must be assessed through both its correctness conditions and its added cost.

At position 1, where the first chunk was corrupted, the combined policy's total elapsed-time differences were +0.066 seconds for truncation and +0.009 seconds for content modification. These differences were also close to zero. Appendix E distinguishes absolute times conditional on success from the cost of all recorded runs.

Table 5. Post hoc analysis of total elapsed time T for correct recovery

Fault	Pos.	Full restart	Syntax checks	Combined	Runs per policy
Stop only	1	104.01	87.73	87.97	48
Stop only	3	145.20	87.80	87.93	48
Truncated JSON	1	103.82	103.90	103.80	48
Truncated JSON	3	145.36	103.68	103.95	48
Text edit	1	103.77	N/A	103.81	48
Text edit	3	145.24	N/A	103.76	48

Times in Table 5 are in seconds. Each value is the equally weighted mean of replicate medians for the 16 recordings. Under content modification, all 48 syntax-based runs at each position were correctness failures, so success-conditional timing is marked N/A. The cumulative total elapsed time of these 96 runs, 8,417.51 seconds, is included in overall cost. The paired differences in Table 4 use a different aggregation order and should not be equated with differences between columns in Table 5.

3 Storage-Boundary and Development Integration Observations

All 16 E3a file cases produced the expected storage states and policy decisions. In three cases where a stale valid artifact and its matching hash-validation record remained together, syntax-based and output-hash-only policies each

permitted three incorrect reuses. Full restart and combined validation permitted zero incorrect reuses in this case set. These observations show that file selection based on an output-hash match can still reuse an artifact from a different task. Appendix C lists file states at each storage boundary and their case identifiers. E3a file contents and execution-condition labels are synthetic; the file-writing process is what was actually terminated. These cases are distinct from the two development integration jobs that interrupted actual speech inference.

Using a separate public development meeting, we first generated donors with another model and another inference setting, then executed two integrated ASR interruption-and-recovery jobs. The first performed inference for one chunk, received an actual SIGKILL, substituted an artifact from another model, and recovered using combined validation. The second performed inference for three chunks, received SIGKILL, substituted an artifact from another setting, and recovered through full restart. Both jobs matched the reference transcript and structured output. They are development cases demonstrating interruption-to-recovery integration under different conditions.

Table C2 provides implementation and resource-planning evidence from these two runs. Each re-inferred 120 seconds and depended on the same development meeting. There were no unexpected execution failures or timeouts. Across 76 service-monitoring samples, no deterioration was observed in services that were healthy at baseline. Costs were recorded separately as pre- and post-interruption inference time and host elapsed time.

VI Discussion and Threats to Validity

1 Interpretation of the Research Questions

RQ1 shows that being able to parse a file is different from preserving its transcript content. Syntax checks alone did not detect altered content when valid JSON structure was retained. For RQ2, reuse of valid chunks reduced recomputation and post-fault latency, but provided almost no benefit when processing had to restart from the first chunk. Together, these findings indicate that recovery policies should be evaluated for result preservation as well as termination, and that cost should be interpreted in relation to the chunks that can actually be reused.

The syntax-based policy is a minimal-functionality control, and its failure under content modification is expected from the scope of its checks. In particular, corruption that changes only text might also be detected through consistency checks across fields. E1 should be interpreted as evidence about the limits of basic validation and recovery cost. The effects of the hash and execution-condition checks combined in E1 must be separated in the frozen supplementary experiment containing a hash-only comparator. Differences from existing methods require a separate comparison under equivalent conditions.

2 Scope of Metrics and Reclassification of Earlier Results

Transcript preservation is evaluated against the same model's uninterrupted output and may therefore preserve the model's original recognition errors. In contrast, word error rate (WER) and character error rate (CER) assess recognition quality against a reference transcript. Diarization error rate (DER) requires speaker-time references and specified evaluation intervals and overlap handling. Korean recognition, diarization, and summarization scores in the earlier manuscript were classified as results requiring revalidation of their scoring procedures.

In the earlier PoC, automatic speaker-count agreement was 0/8 and cross-chunk speaker-linking DER was 74.05%. These results are retained as diagnostic records under their original conditions and distinguished from E1's recovery evaluation. Further validation of speaker-linking performance is required before judging applicability to actual person identification or automatic assignment of responsibilities.

3 Generalization and Statistical Interpretation

The evaluation uses the first 120 seconds of 16 recordings from four public English meeting series. Generalization to middle and later portions of long meetings, cross-chunk contextual dependence, and Korean construction meetings is therefore limited. We report descriptive statistics in view of dependence within recordings and series and the three technical replicates per condition. Population confidence intervals, significance tests, and tail-latency estimates require independent data and further measurements.

Another limitation is that prior inspection of test data and intermediate E1 results influenced the supplementary design. Subsequent validation on an independent corpus should fix the model and conditions before evaluation and document any previous access to the data and reference annotations.

The injected CORRUPTED OUTPUT string and half-file truncation are reproducible artificial faults. The results describe policy responses to these specified faults. Estimating operational fault distributions and reliability requires additional evidence on field occurrence rates. Because the supplementary experiment uses the same 16 recordings, validation on an independent corpus remains a separate task.

4 Operational and Security Limitations

Absolute latency measured on a shared host with CPU limits is affected by hardware, load, cache state, and processing order. Service monitoring was an operational control that compared current states with previously recorded states. Effects on request latency and availability require separate measurements.

E1 and E3a observe process termination under trusted validation records and a trusted controller. Protection and durability against simultaneous forgery of artifacts and records, host compromise, power loss, and storage-device damage require further validation. Deployment also requires procedures for input and model distribution, privilege separation, log retention, key management, and incident response.

5 Reproducibility and Research Ethics

Per-run results, errors, partial files, and frozen specifications are retained in their original form, including failure cases. The main protocol is identified as KSCI-ARTIFACT-RECOVERY-MAIN-20261026, and the relationships among hashes, inputs, code, and schedules are provided in a separate execution manifest. Before distributing reproduction materials, the terms for data, code, and models must be checked separately, and internal connection information and credentials must be removed.

Execution evidence is currently retained as local reproduction material. A publicly accessible location and independent external reproduction remain to be established. The application context and system

configuration derive from the earlier conference PoC [17]; the present study adds the fault protocol, experiments, and analysis.

6 Total Cost and Conditions for Comparing Existing Methods

An operational total-cost comparison must combine normal-execution cost, pre-interruption inference, and post-fault inference, model loading, validation, storage, and control costs under the same measurement boundaries. This paper uses E1 post-fault latency as its primary measure and reports a post hoc analysis of total elapsed time, including pre-interruption cost, in Appendix E. Estimating average savings in operation also requires measuring fault frequency.

Comparisons with existing workflows must provide the same audio, model, settings, chunk dependencies, and resource limits. First, the output-hash-only and combined-validation policies should be compared to isolate the incremental effect of execution-condition checks. Existing tools should then be configured with the same information and reuse units to compare reuse of valid files, detection of content corruption, handling of valid artifacts from other tasks, and recovery cost. When correctness is equal, added normal-execution cost and the extent of recomputation should be analyzed. These comparisons and independent long-form validation are needed to extend the scope of the present conclusions.

VII Conclusion

This study develops artifact completeness and checkpoint recovery within an on-premises meeting pipeline for construction enterprises. It separates transcript reuse into artifact syntax, byte integrity, and execution-condition matching, and evaluates policies through fault injection with the same model, inputs, and storage format. All 912 runs terminated, yet all 96 runs combining content modification with syntax-based reuse committed incorrect transcripts. This demonstrates the need to assess transcript preservation alongside execution completion. Validation-based reuse preserved transcripts within this limited matrix and reduced recomputed audio and post-fault latency only when valid checkpoints remained.

Under the specified fault conditions, content validation contributed to detecting corruption that preserved file syntax, while the size of cost savings depended on the extent of reusable valid chunks. Operators can therefore consider both validation scope and recovery cost when choosing a policy. Further validation will focus on an output-hash-only comparator and existing workflows configured under equivalent conditions, followed by assessment on independent long-form data.

References

[1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, Robust Speech Recognition via Large-Scale Weak Supervision, Proceedings of the 40th International Conference on Machine Learning, PMLR, Vol. 202, pp. 28492–28518, 2023. https://proceedings.mlr.press/v202/radford23a.html

[2] M. Bain, J. Huh, T. Han, and A. Zisserman, WhisperX: Time-Accurate Speech Transcription of Long-Form Audio, INTERSPEECH 2023, pp. 4489-4493, 2023. https://doi.org/10.21437/Interspeech.2023-78

[3] H. Bredin, pyannote.audio 2.1 Speaker Diarization Pipeline: Principle, Benchmark, and Recipe, INTERSPEECH 2023, 2023. https://doi.org/10.21437/Interspeech.2023-105

[4] E. N. Elnozahy, L. Alvisi, Y.-M. Wang, and D. B. Johnson, A Survey of Rollback-Recovery Protocols in Message-Passing Systems, ACM Computing Surveys, Vol. 34, No. 3, pp. 375-408, 2002. https://doi.org/10.1145/568522.568525

[5] M.-C. Hsueh, T. K. Tsai, and R. K. Iyer, Fault Injection Techniques and Tools, Computer, Vol. 30, No. 4, pp. 75-82, 1997. https://doi.org/10.1109/2.585157

[6] Bazel Contributors, Remote Caching, Bazel Documentation. https://bazel.build/remote/caching (accessed 2026-10-29)

[7] P. Di Tommaso, M. Chatzou, E. W. Floden, P. P. Barja, E. Palumbo, and C. Notredame, Nextflow Enables Reproducible Computational Workflows, Nature Biotechnology, Vol. 35, pp. 316-319, 2017. https://doi.org/10.1038/nbt.3820

[8] Nextflow Contributors, Caching and Resuming, Seqera Documentation. https://docs.seqera.io/nextflow/cache-and-resume (accessed 2026-10-29)

[9] Iterative, Running Pipelines, DVC Documentation. https://doc.dvc.org/user-guide/pipelines/running-pipelines (accessed 2026-10-29)

[10] Seqera, Getting Started with Data Lineage, Nextflow Documentation, experimental feature introduced in version 25.04. https://docs.seqera.io/nextflow/tutorials/data-lineage (accessed 2026-10-29)

[11] Snakemake Contributors, Between Workflow Caching, Snakemake 8.20.4 Documentation. https://snakemake.readthedocs.io/en/v8.20.4/executing/caching.html (accessed 2026-10-29)

[12] T. S. Pillai, V. Chidambaram, R. Alagappan, S. Al-Kiswany, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications, OSDI 14, pp. 433-448, 2014. https://www.usenix.org/conference/osdi14/technical-sessions/presentation/pillai

[13] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, Zero Trust Architecture, NIST SP 800-207, 2020. https://doi.org/10.6028/NIST.SP.800-207

[14] AMI Consortium, AMI Corpus. https://groups.inf.ed.ac.uk/ami/corpus/ (accessed 2026-10-29)

[15] pyannote Contributors, AMI Diarization Setup, a fork of the BUT Speech@FIT setup, revision 67c2d539286e89f68952d5dcf83912bd9f01dfae. https://github.com/pyannote/AMI-diarization-setup/tree/67c2d539286e89f68952d5dcf83912bd9f01dfae

[16] S. H. Hurlbert, Pseudoreplication and the Design of Ecological Field Experiments, Ecological Monographs, Vol. 54, No. 2, pp. 187-211, 1984. https://doi.org/10.2307/1942661

[17] J. Kim, J. Yun, J. Lee, and K. Kim, A Secure On-Premises Meeting Intelligence Pipeline Using Edge AI for Construction Enterprises, Proceedings of the 74th Summer Conference of the Korea Society of Computer and Information, Jeju, Republic of Korea, July 2026.

Appendix A Evaluation Scope and Further Validation

Tables 3 and 4 are calculated from the completed E1 experiment, while Table C2 reports two completed integration jobs using a separate public development meeting. Results for each protocol are aggregated separately using that experiment's denominator. The results in this paper are limited to this evidence, completed and reviewed when the manuscript was finalized on October 29, 2026.

An output-hash-only comparator, artifacts generated from different inputs, models, and settings, and independent-corpus validation are subjects of follow-up work. Results from the ongoing supplementary main experiment are managed separately from the completed E1 and development cases.

Quality evaluation against official references is a separate task. Equivalence between the existing custom-normalization scores and the original distribution's scoring procedure must be established, and speaker-attribution evaluation requires speaker-time references. This paper evaluates transcript preservation relative to an uninterrupted reference run with the same input and model.

Aggregation first computes paired differences for each replicate, then the median for each recording, and finally the mean of the 16 medians. For example, if the three differences for one recording are -10, -12, and -30 seconds, that recording's representative difference is -12 seconds (hypothetical values for illustration). Subtracting policy medians computed first may yield a different value. The absolute-time means and paired differences in Appendix E follow their respective aggregation orders.

Appendix B Variation Across Recordings in the Existing Results

Table B1 is a post hoc descriptive analysis of recording-level paired differences using the estimator and inclusion rules of Table 4. Ranges are the observed minima and maxima; series means weight each of the four recordings in a series equally.

Table B1. Distribution of latency differences for combined validation and means by series

Fault	Min-max (s)	EN2002	ES2004	IS1009	TS3003
Stop only, pos. 1	-29.131 ~ -8.801	-11.757	-16.722	-14.513	-21.251
Stop only, pos. 3	-73.841 ~ -39.440	-49.238	-56.353	-57.140	-66.518
Truncated JSON, pos. 1	-1.287 ~ 0.613	0.037	0.416	0.030	-0.487
Truncated JSON, pos. 3	-49.834 ~ -28.943	-37.416	-40.007	-42.350	-45.693
Text edit, pos. 1	-0.798 ~ 1.117	0.158	-0.465	0.394	0.007
Text edit, pos. 3	-51.253 ~ -28.901	-37.542	-39.787	-42.928	-45.498

Conditions containing both small positive and small negative values indicate variation around zero relative to full restart. The three technical replicates characterize variation under the specified execution conditions.

Appendix C Execution Specification and Storage-Boundary Observations

E1 uses 16 kHz mono PCM AMI Mix-Headset input, 30-second chunks, OpenAI Whisper large-v3-turbo, two CPU threads, English, temperature 0, beam size 1, disabled fp16, and no conditioning on previous chunk transcripts. Containers are allocated the equivalent of 2 CPUs and 8 GiB memory, with GPU and external network access disabled. Full identifiers for the image, model, code, inputs, and schedule are retained in the accompanying execution_manifest.json. Verification of Python, PyTorch, and Whisper package versions against the frozen image, as well as external reproduction, remains follow-up work.

E3a examines the same eight positions from two initial states: an empty state and a state containing a valid artifact-record pair generated under different input conditions. The file-writing process is actually terminated at the first seven positions, while the last is an uninterrupted control, producing 16 cases in total. Temporary files are excluded from reuse; the artifact and validation record in Table C1 refer to the states remaining at their final paths. New files are synthetic data with the currently expected execution conditions; old files are synthetic data for a different input.

Table C1. Expected and observed states at storage boundaries. Entries show artifact / validation-record states.

Case IDs	Interruption / control point	Empty start	Old pair
00 / 08	Before opening the artifact temporary file	None / none	Old / old
01 / 09	After partially writing the artifact temporary file	None / none	Old / old
02 / 10	After syncing the artifact temporary file, before replacement	None / none	Old / old
03 / 11	After artifact replacement, before writing its record	New / none	New / old
04 / 12	After partially writing the validation-record temporary file	New / none	New / old
05 / 13	After syncing the record temporary file, before replacement	New / none	New / old
06 / 14	After record replacement, before the completion marker	New / new	New / new
07 / 15	Uninterrupted control	New / new	New / new

Case numbers correspond to archived records e3a-plan-00 through e3a-plan-15. All 16 cases matched the expected states in Table C1. In cases 08, 09, and 10, the old file and its matching hash-validation record remained together, so syntax and output-hash checks alone could not prevent incorrect reuse. Combined validation rejected them after identifying a mismatch with the currently expected input conditions. These results demonstrate the file-selection rules at the specified storage boundaries.

The two development ASR integration jobs used the same public development meeting. In the different-model case, the donor was generated with tiny.en and the recovery target was large-v3-turbo. In the different-setting case, an artifact generated with large-v3-turbo at beam size 2 was applied to a target task with beam size 1. Interruption occurred after inference for the first and third chunks, respectively, and the policies differed: combined validation and full restart. Table C2 records actual interruption-recovery integration and execution cost for these two development cases.

Table C2. Cost breakdown of the development ASR integration jobs

Development case	Pre-stop inference (s)	Recovery inference (s)	Total inference (s)	Host (s)
Different model; pos. 1; combined	25.811	116.741	142.552	155.433
Different setting; pos. 3; full restart	85.552	117.222	202.775	215.426

Appendix D Relationship Between the PoC Materials and This Experiment

Fig. 6 presents the PoC equipment to illustrate the application context of an enterprise meeting-transcription system. It shows the Raspberry Pi 5 board and the development setup connecting a GIGABYTE DGX Spark, an audio interface, and capture devices.

The project interim report identifies speech capture for secure environments, transcription and speaker differentiation, and meeting summarization as development objectives. It records verification of a meeting-record list and screens displaying participant-specific transcripts and summaries. These materials illustrate an application in which transcript artifacts are subsequently viewed and summarized. Reusing an incorrect intermediate artifact can propagate errors into downstream processing, so this study confines its evaluation to transcript-file reuse decisions and recovery cost.

The quantitative evaluation in the main text uses public AMI recordings. The PoC photographs illustrate the application environment connecting speech-capture devices and computing equipment.



(a) (b)

Fig. 6. PoC equipment: (a) Raspberry Pi 5 board; (b) GIGABYTE DGX Spark and connected peripherals.

Source: PoC equipment photographs supplied by the authors and the Collaboration Project Interim Report (August 17, 2026).

Appendix E Post Hoc Analysis of Total Elapsed Time

This post hoc analysis reread the 912 archived result.json files after E1 ended. The 48 runs per condition are three replicates for each of 16 recordings. The values below are in seconds: among runs returning correct results, replicate medians are calculated for each recording and then averaged with equal weights across all 16 recordings. Policies are ordered as full restart, syntax-based reuse, and combined validation. Values outside parentheses are post-fault latency L; those inside parentheses are total elapsed time T, including pre-interruption inference.

Stop only, position 1: 83.63 (104.01), 67.39 (87.73), 67.64 (87.97).

Stop only, position 3: 83.52 (145.20), 26.19 (87.80), 26.19 (87.93).

JSON truncation, position 1: 83.55 (103.82), 83.59 (103.90), 83.49 (103.80).

JSON truncation, position 3: 83.49 (145.36), 42.06 (103.68), 42.12 (103.95).

Content modification, position 1: 83.48 (103.77), N/A, 83.49 (103.81).

Content modification, position 3: 83.53 (145.24), N/A, 42.09 (103.76).

Under content modification, all 48 syntax-based runs at each position returned incorrect transcripts, so timing conditional on success is marked N/A. These runs remain included in the overall results and cost totals. The sum of total elapsed time for the 96 runs is 8417.51 seconds.

The sum of per-run T over all 48 no-fault controls and 864 fault-injection runs is 95043.46 seconds (26.40 hours). This is cumulative elapsed time over execution intervals that include pre-interruption inference and recovery. SHA-256 identifiers for the raw records, condition-level and recording-level aggregates, and analysis code are retained as separate reproduction materials.

For common instrumentation in E1, every policy writes validation records. Isolating the cost of writing these records and of the complete validation mechanism requires a no-fault comparison that changes only validation functionality while keeping the storage procedure identical.

Appendix F Transcription and Summarization Service Interface

Fig. 7 is a development screenshot of the PoC service using lecture audio. Users inspect the chronological transcript on the left and the summary on the right. It illustrates an application flow in which stored transcripts serve as inputs to subsequent summarization.

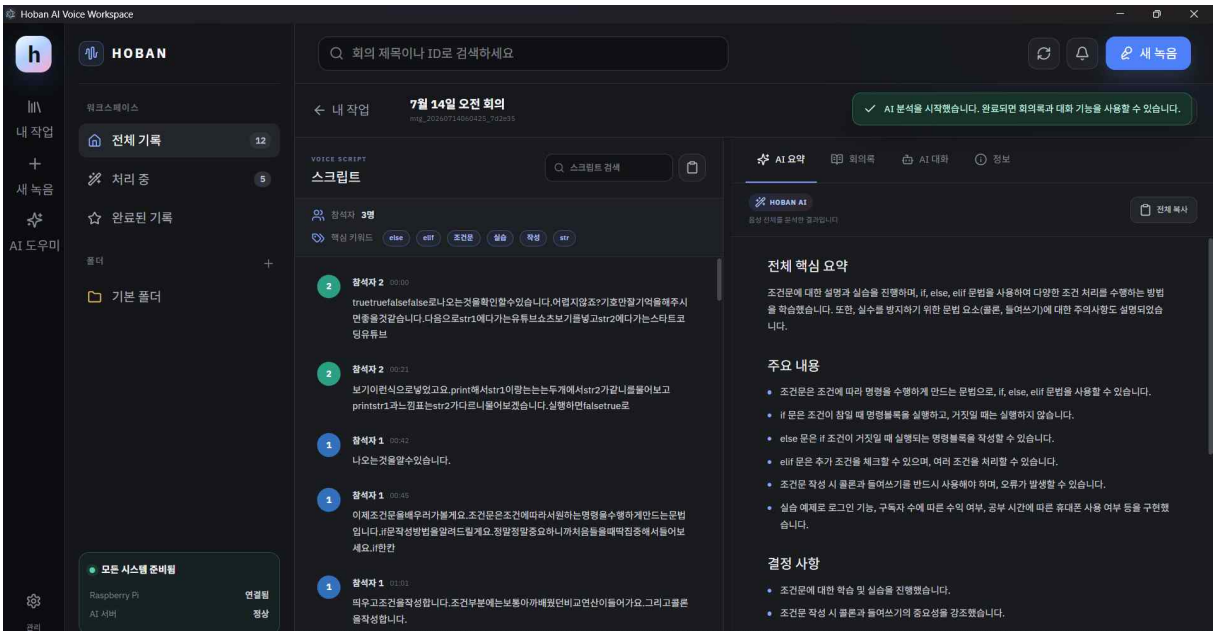


Fig. 7. Development screenshot of the PoC transcription and summarization service using lecture audio.

Source: Service screenshot supplied by the authors (development material from July 2026).

Authors



Jaehwan Kim is a Ph.D. candidate researcher at aSSIST (Seoul School of Integrated Sciences and Technologies), Seoul, Republic of Korea. His work in this study covers the overall system design, pipeline implementation, empirical measurements, and manuscript preparation. His research interests include on-premises meeting intelligence and edge AI.



Jaeseong Yun is an AI Engineer at XAIKOREA Inc., Seongnam, Republic of Korea. His work in this study covers speech-AI processing, vLLM serving, and concurrency for meeting summarization.



Jongseok Lee is the Infrastructure Team Leader at RAD CNS Co., Ltd., Republic of Korea. His work in this study covers the two-node GPU infrastructure and service orchestration.



Kyungmin Kim is a Principal Researcher in the Technical Support Team at Security Platform, Republic of Korea. His work in this study covers the security architecture, AES-256-GCM protection, and Vault KEK integration.